OKD4 Cluster

Openshift, Kubernetes and the Cluster Console

Richard McCreadie

How Do I Use it?

What you need to do will define what you need to learn to use the cluster, you can think of it as having 4 levels, where you incrementally need to understand more of how the cluster works as you need to undertake more complex work

Level 0: Understanding your account, resour

You are Here

Level 1

- Cluster users who just need to get up and running with Python/Jupyter or need remote execution with VSCode are recommended to use the Launcher tool

Level 2

- For scenarios where you need to deploy a wider range of existing services, you will need to learn the basics of Containers and Kubernetes. This will allow you to deploy pre-built container solutions from platforms like Dockerhub or more complex services via OperatorHub.

Level 3

- For running batch experiments (queuing many containers to run concurrently) you will need to learn how to interact with the cluster via the command line tool (oc)

Level 4

- If you need to create a new development environment from first principles or custom applications/demos, then you will need to learn how to produce new containers via Builds, and it is recommended to learn how to create continuous integration pipelines.

Going Deeper…

University of Glasgow

These more advanced slides are here for people that need to go beyond Launcher

- This usually means that you are working on a more complex application that involves multiple services, need deeper customisation of your sandbox, or are working towards setting up scripted batch jobs

In these slides we will effectively introduce how to do what launcher does manually, which will give you the tools to deploy and customise your own containers, in particular, the topics we will cover are:

- Containers and how they work
- Kubernetes and how we can tell the cluster scheduler to run a container
- Application replication and roll-outs of new versions
- Configuration of external network access to a container
- Configuration of mounted persistent storage to containers
- Introduce the Cluster Console

More on Containers

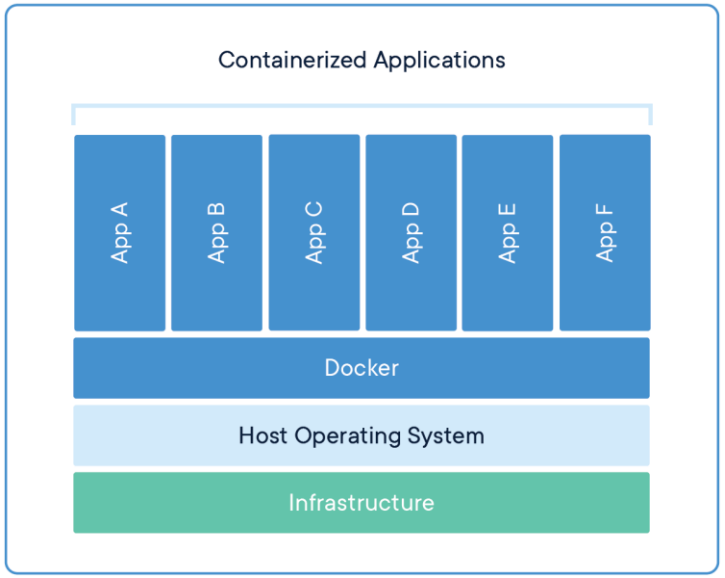
What is a Container?

In the previous tutorial we introduced the idea of the IDA Container that users tend to start with

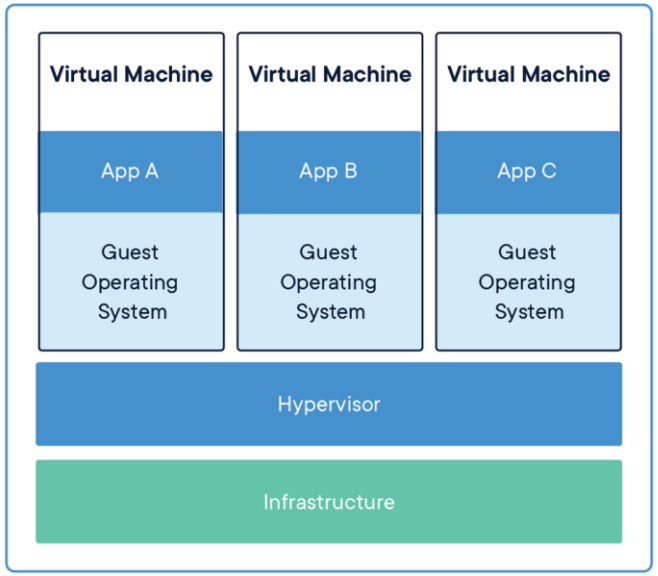
But what is a Container?

Containers are lightweight alternatives to virtual machines

- Shares the OS with host system while still isolating software running inside
- Created from "images", which are layers of modifications on top of a "base image"
- Images are made available through image repositories



Container Stack



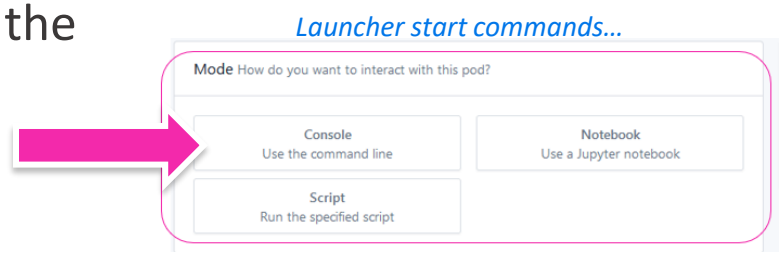
Virtual Machine Stack

https://www.docker.com/resources/what-container

Containers

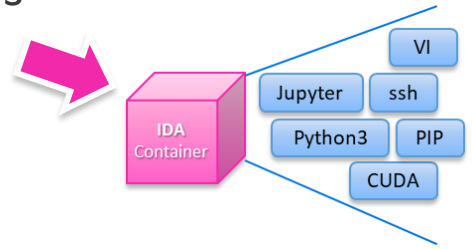
- You can consider a container to be a sandbox environment where you can run your code

- Each container runs a single command from the command line, and the container is destroyed once the command exits
  - This command may be a script that carries out a sequence of actions
  - Or it may start an application that will run continuously



- Any changes made to a container environment is lost when the container exits
  - So, all program output must be written to external mounted file stores during run-time, hence the emphasis on having a personal persistent folder for storage

- The container image defines what software is available to run, i.e. the libraries that are installed initially, such as python or nvidia-smi
  - Your code and data will typically be mounted, not baked into the image



Why do we need Containers?

University of Glasgow

One of the core issues when managing hardware is that it needs to be shared amongst multiple users with different types of workloads

This raises important challenges

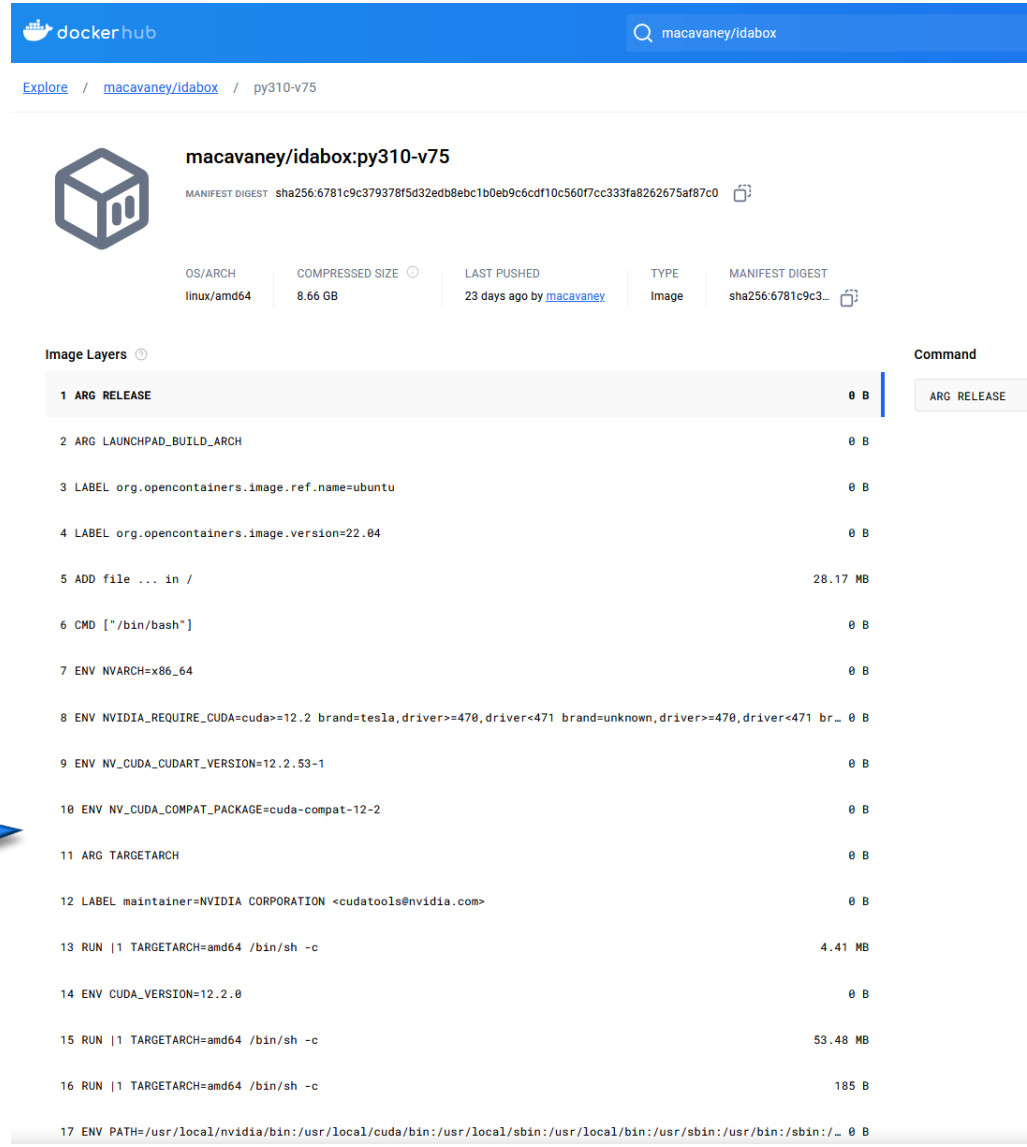
- How do we avoid library versioning and incompatibility issues amongst the software that is running?
- How do we make sure each application has the right number of resources?
- How do we make applications portable, such that they can be run anywhere?

Containers solve these issues by packaging your application into a personal sandbox with only your libraries and code, which is also portable

Container Image

- The portable form of a container is the container image
  - ➢ This is a blueprint for a (typically) linux environment
- There are a small number of 'base' images, which act like a clean install of an operating system
  - ➢ These are designed to be as light-weight as possible
- All other images you find on image repositories are recorded modifications (known as layers) on top of a base image

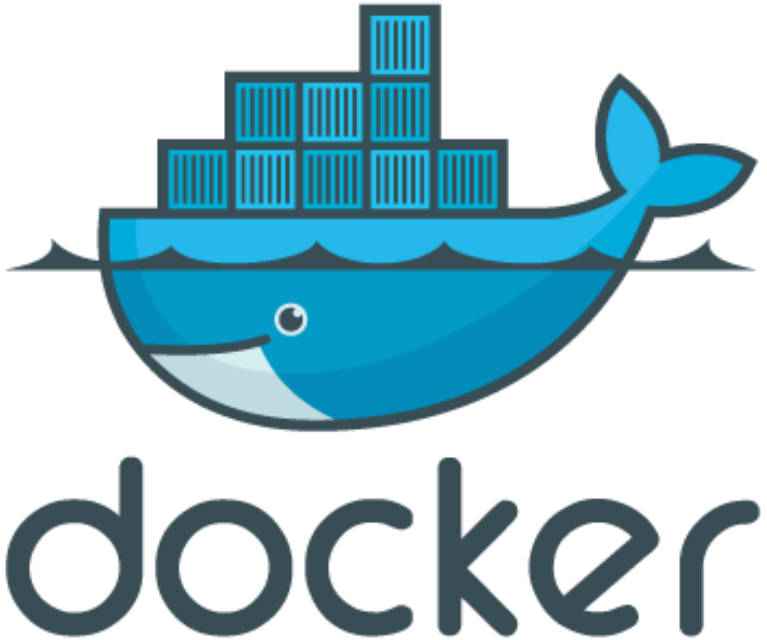
Here is (part) of the IDA Container layers



An Aside on Docker

University of Glasgow

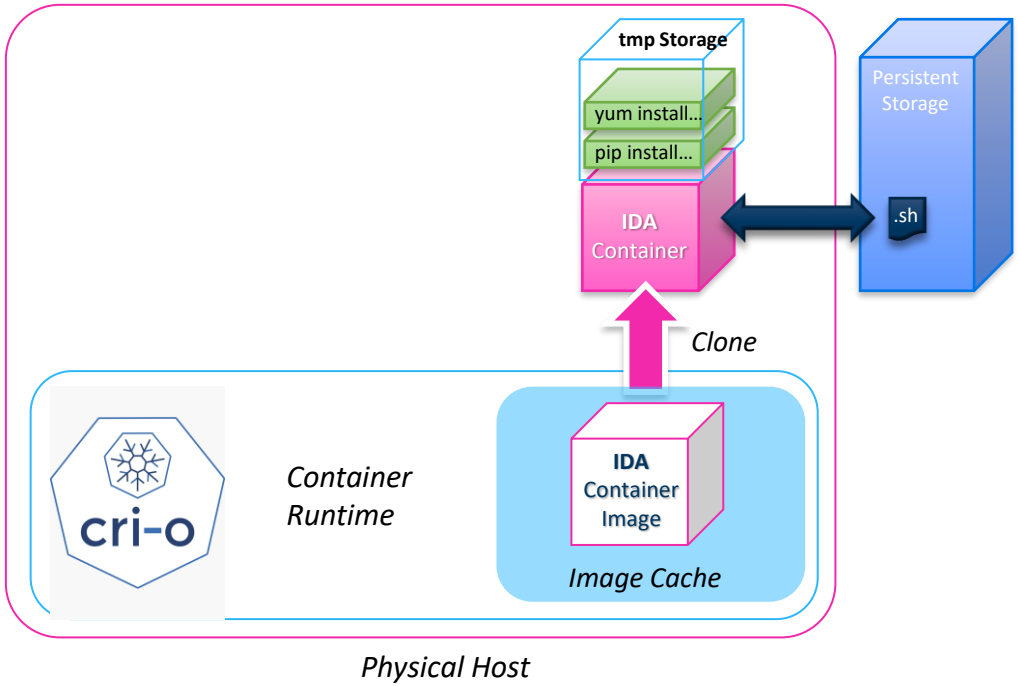
- Docker is a company that produces commercial solutions for container creation and management
- There are many types of containers, however, Docker designed the defacto-standard type, namely: Docker Containers
- This is backed by one of the largest container image repositories – Dockerhub.io
  - Provides ready-to-deploy images for the majority of application types
- OKD4 is compatible with Docker containers, but does not use docker engine to execute those containers



Understanding Container Modification

University of Glasgow

- When you start a container, this deploys it onto a container runtime (CRI-O in our case) on a target machine
- The container is generated by cloning a container image
  - This image will be downloaded from an external image repository (e.g. DockerHub) to the container-runtime's local image cache
- The container executes its run command with arguments
  - This is usually defined by the container, but can be overwritten
- Installing libraries or Python packages updates the running container, not the container image
  - If the container exits for any reason, those modifications are lost
  - This is because installations are stored in temporary storage on the host, and cleaned up with the container
- Attached persistent storage can hold data between container instances
  - It is recommended to create a script with install commands in there that you can quickly run to layer-in any needed libraries for your work



Kubernetes

Allocating Containers to Machines

University of Glasgow

You know something about what a container is, but we are working with a cluster environment where there are multiple machines, at some point our container needs to get allocated to a physical machine/host…. so how does that happen?

Our cluster has a unified management system that handles work allocation

- What Launcher does is communicates your request to start an IDA container (with a specified set of resources) to this management system, which handles the rest

It is this management system that the cluster is named after, it is called

… *sort of*, we are actually using the open-source version called OKD, with major version 4, hence **OKD4**

OpenShift / OKD

University of Glasgow

So… what is OpenShift?

- It is a very opinionated deployment of ***Kubernetes***.
- OpenShift pre-installs lots of useful services like a container image store, dashboards, container builds and more

Then what is Kubernetes?

- It is an open-source system for automating deployment, scaling, and management of containerized applications.

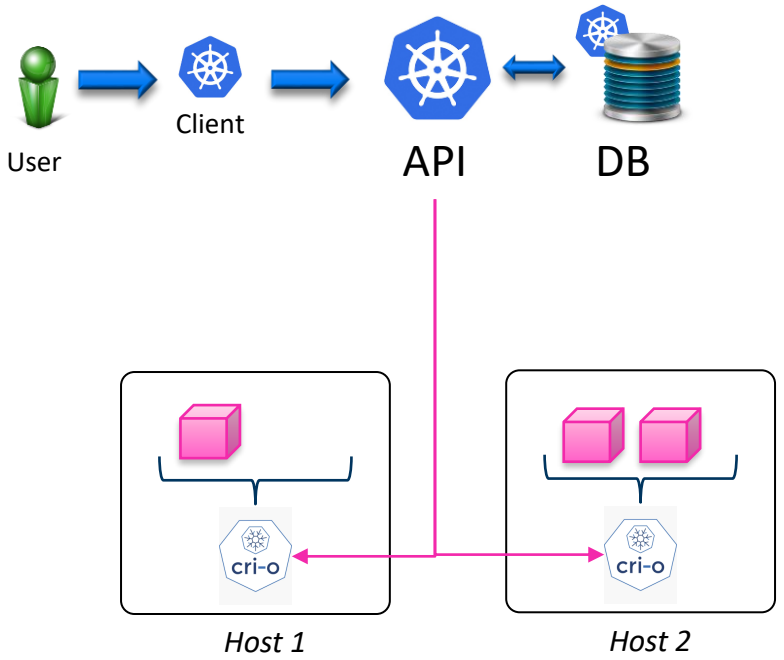
kubernetes

Kubernetes

We won't get into exactly how Kubernetes works here, but here are the basics

The core of Kubernetes is the API Service

- Facilitates user registration of state specifications (referred to as Kubernetes Objects) via client
  - These state specifications are stored in a database
- Regularly compares cluster state to the stored state specifications, if they differ, it will try to take actions to correct this
  - Actions orchestrate CRI-O instances on the local machines (which can run containers)



State Specifications / Kubernetes Objects

University of Glasgow

What is a state specification?

- It is a user declaration of a desired state they want the cluster to be in

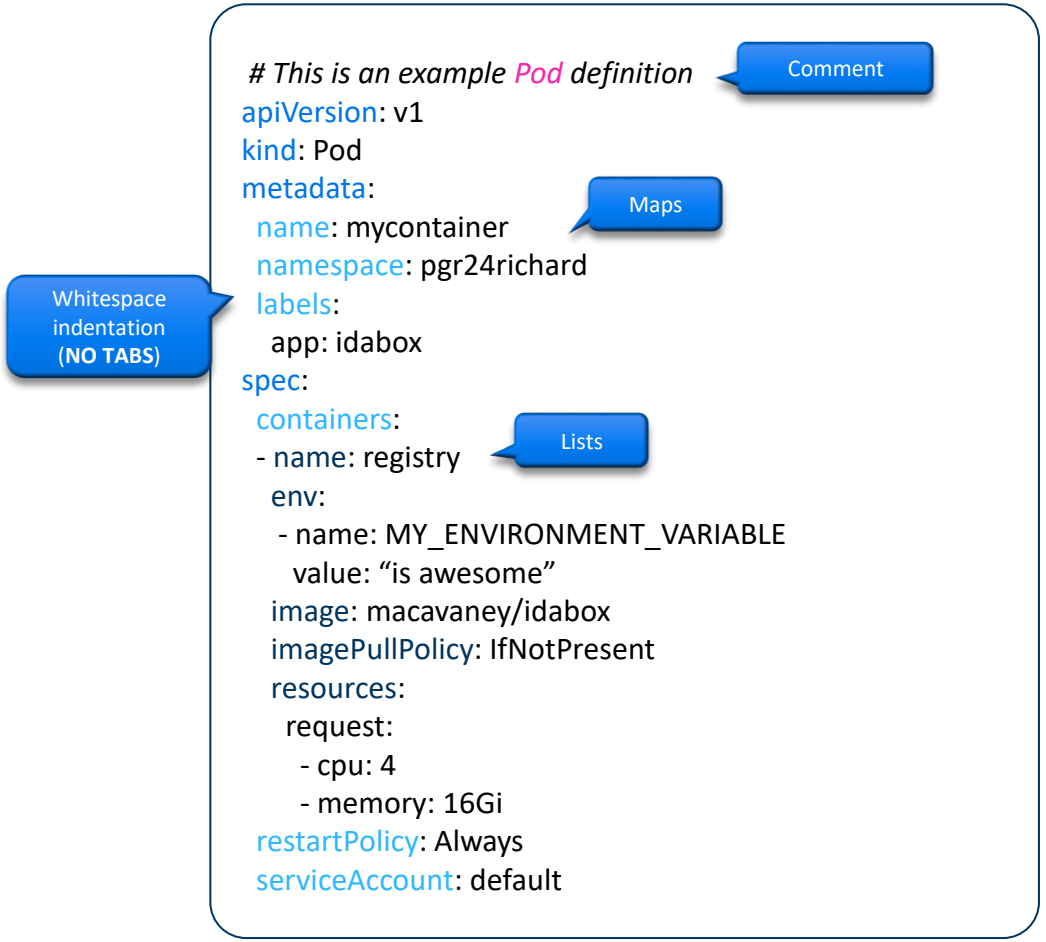
This is easier to explain with an example, let's say that I want to start an IDA Container

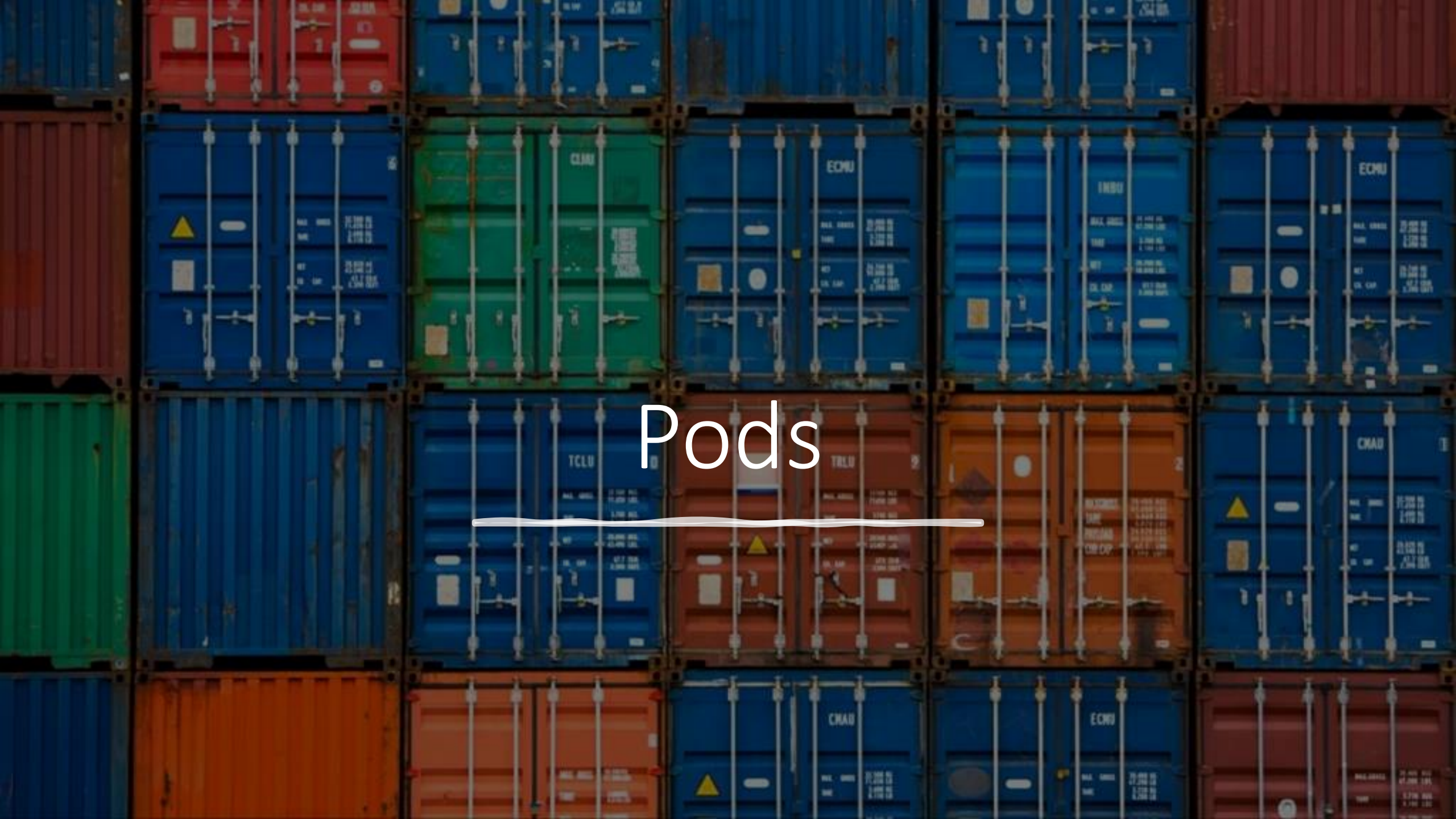
- I would write a Kubernetes Object file specifying that I want the cluster to have one IDA Container running in my personal project space
- I register this Kubernetes Object with the Kubernetes API, which stores it
- The Kubernetes API regularly reads my stored Kubernetes Object and checks whether the cluster state matches my desired state (is my container running?)
  - The first time it checks, it will discover that it is not
  - It will then try and schedule the missing container by starting it on one of the machines via CRI-O
  - Future checks it will detect my container is running and so the cluster state will match my desired state (and so the Kubernetes API will be happy and do nothing)
  - If my container dies for any reason, the cluster state and my desired state will diverge, resulting in the Kubernetes API trying to re-schedule my container

Kubernetes Object File

So… what does a Kubernetes Object File look like?

- It's a plain text file in YAML format
- YAML is a human-readable format that targets many of the same communications applications as XML but has a minimal syntax





Pods

Pod

A Pod is a type of Kubernetes Object that defines one or more Containers to run

- It's the foundation for all compute workloads

Containers within a Pod have some key properties

- All containers within a Pod will run on the same physical host
- When scheduling:
  - The sum of the resource requests for all containers is used to determine if a host has sufficient resources to receive that Pod
  - A Pod can express host preferences, e.g. it wants to be on a host with GPUs,
- If any of the containers within a Pod fails
  - The Pod as-a-whole is considered to have failed
  - All remaining containers will be killed
- Containers within a Pod can 'see' (from a network perspective) each other, i.e. are not isolated

Example Pod

University of Glasgow

Lets explore this example...

- A Pod is a type of Kubernetes Object, which defines one or more Containers to run

At the top of any Kubernetes Object there is the **API Version**, this tells your client where to send this object when communicating with the API, **v1** just means the default endpoint

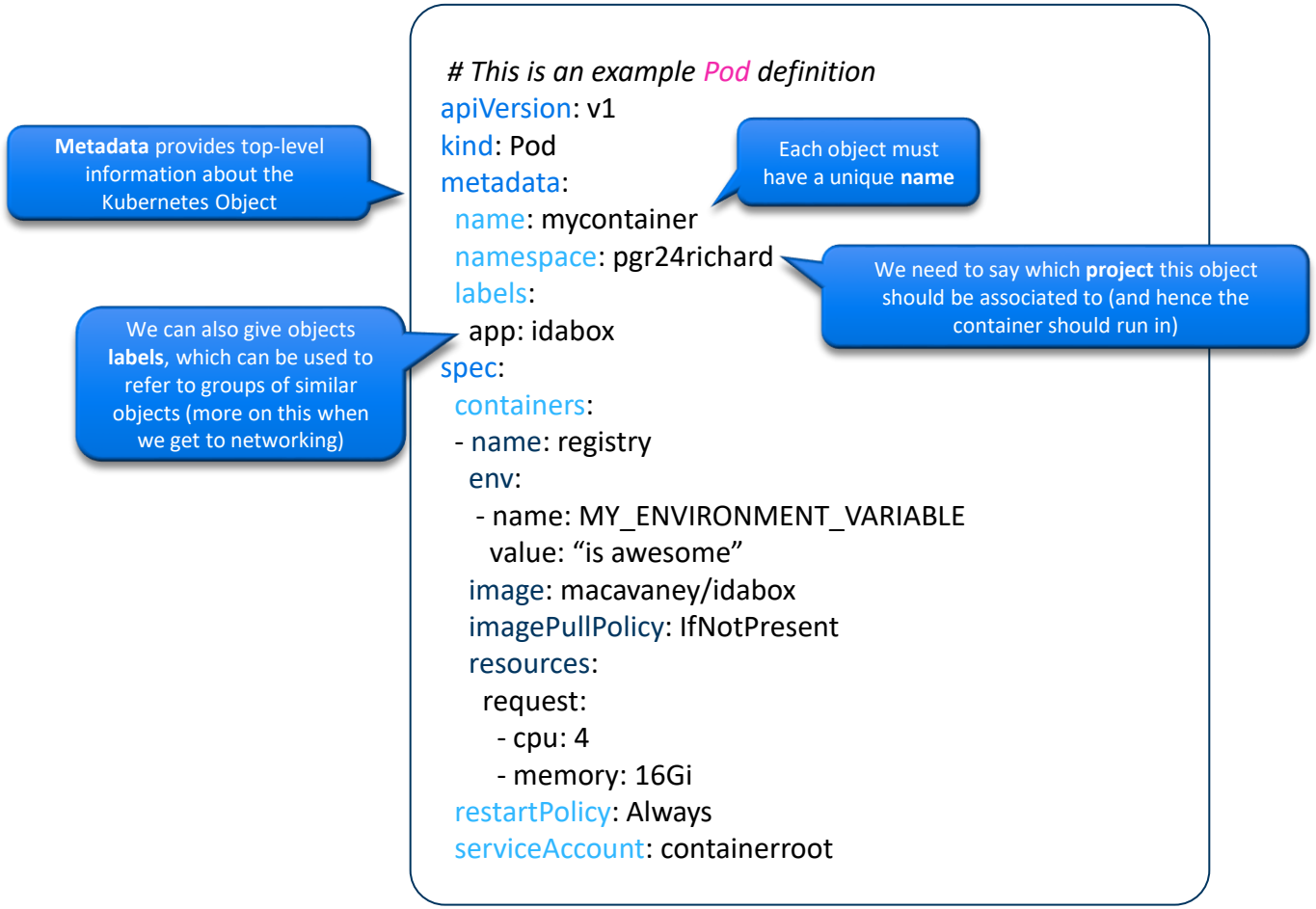
```
# This is an example Pod definition
apiVersion: v1
kind: Pod
metadata:
  name: mycontainer
  namespace: pgr24richard
  labels:
    app: idabox
spec:
  containers:
  - name: registry
    env:
     - name: MY_ENVIRONMENT_VARIABLE
       value: “is awesome”
    image: macavaney/idabox
    imagePullPolicy: IfNotPresent
    resources:
      request:
        - cpu: 4
        - memory: 16Gi
  restartPolicy: Always
  serviceAccount: containerroot
```

We also need to specify the type/**kind** of the Kubernetes Object we are defining, this is used to validate the format of the rest of the file

Example Pod

Lets explore this example...

- A Pod is a type of Kubernetes Object, which defines one or more Containers to run

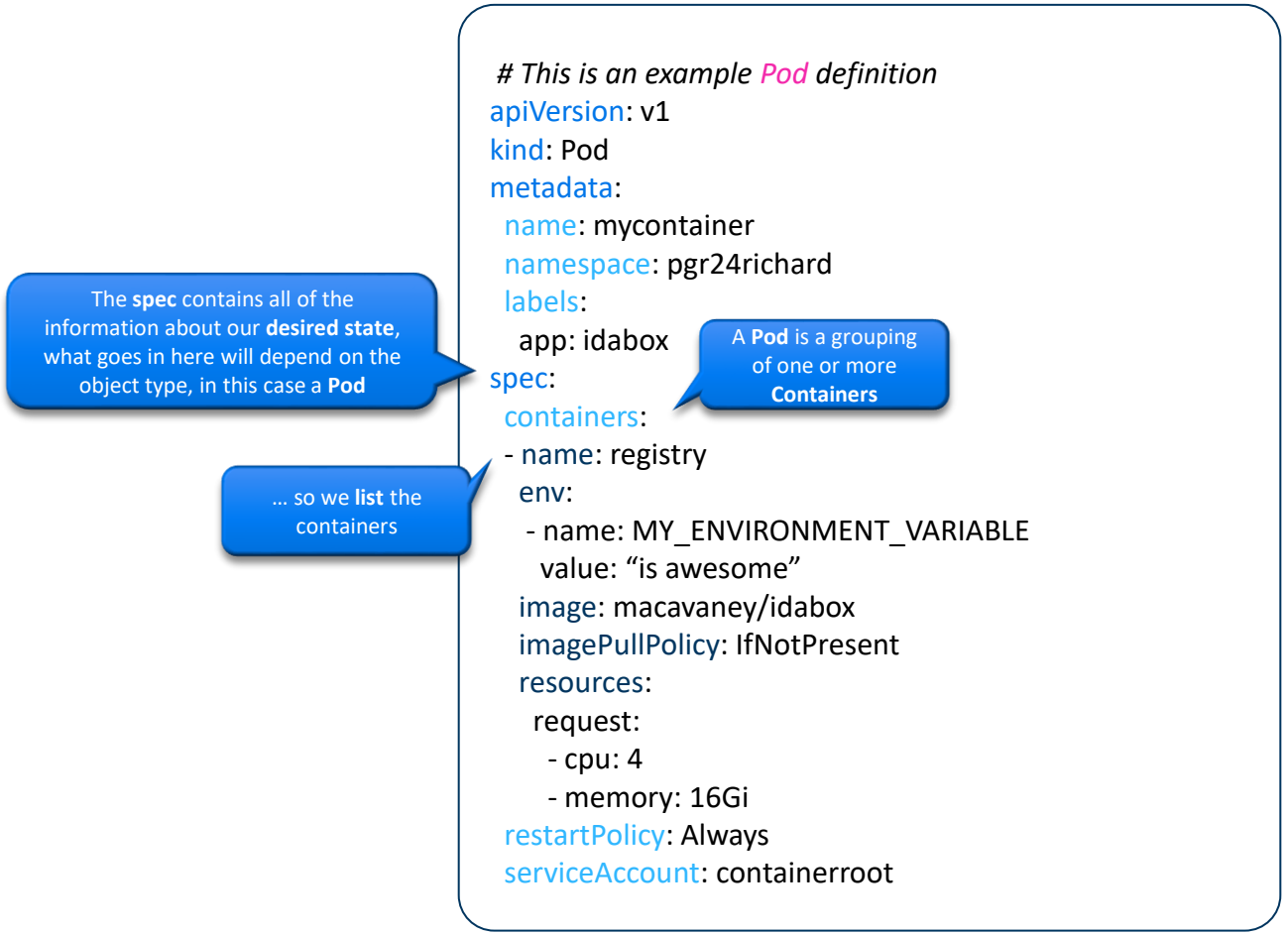


Example Pod

University of Glasgow

Let's explore this example...

- A Pod is a type of Kubernetes Object, which defines one or more Containers to run



Example Pod

University of Glasgow

Let's explore this example...

- A Pod is a type of Kubernetes Object, which defines one or more Containers to run

```
# This is an example Pod definition
apiVersion: v1
kind: Pod
metadata:
  name: mycontainer
  namespace: pgr24richard
  labels:
    app: idabox
spec:
  containers:
  - name: registry
    env:
     - name: MY_ENVIRONMENT_VARIABLE
       value: “is awesome”
    image: macavaney/idabox
    imagePullPolicy: IfNotPresent
    resources:
      request:
        - cpu: 4
        - memory: 16Gi
  restartPolicy: Always
  serviceAccount: containerroot
```

Each container needs its own **name**

We can set environment variables

Example Pod

University of Glasgow

Let's explore this example...

- A Pod is a type of Kubernetes Object, which defines one or more Containers to run

```
# This is an example Pod definition
apiVersion: v1
kind: Pod
metadata:
  name: mycontainer
  namespace: pgr24richard
  labels:
    app: idabox
spec:
  containers:
  - name: registry
    env:
     - name: MY_ENVIRONMENT_VARIABLE
       value: “is awesome”
    image: macavaney/idabox
    imagePullPolicy: IfNotPresent
    resources:
      request:
        - cpu: 4
        - memory: 16Gi
  restartPolicy: Always
  serviceAccount: containerroot
```

This is where to download the container **image** from, if a full url is not provided, it will default to DockerHub

Remember how local CRI-O instances had an internal cache? This tells CRI-O only download the image if it does not have a local copy

Example Pod

University of Glasgow

Let's explore this example...

- A Pod is a type of Kubernetes Object, which defines one or more Containers to run

```
# This is an example Pod definition
apiVersion: v1
kind: Pod
metadata:
  name: mycontainer
  namespace: pgr24richard
  labels:
    app: idabox
spec:
  containers:
  - name: registry
    env:
     - name: MY_ENVIRONMENT_VARIABLE
       value: “is awesome”
    image: macavaney/idabox
    imagePullPolicy: IfNotPresent
    resources:
      request:
        - cpu: 4
        - memory: 16Gi
  restartPolicy: Always
  serviceAccount: containerroot
```

Containers need **resources**...

4 CPU cores

16GB of RAM

Example Pod

University of Glasgow

Let's explore this example...

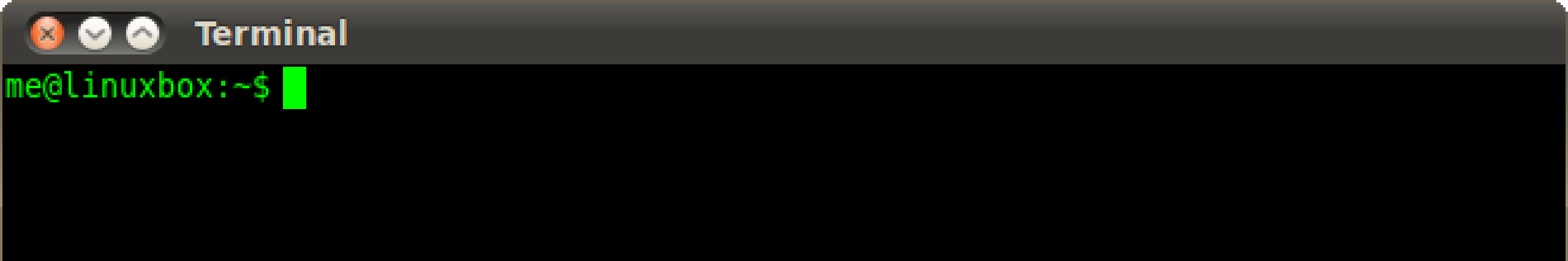
- A Pod is a type of Kubernetes Object, which defines one or more Containers to run

```
# This is an example Pod definition
apiVersion: v1
kind: Pod
metadata:
  name: mycontainer
  namespace: pgr24richard
  labels:
    app: idabox
spec:
  containers:
  - name: registry
    env:
     - name: MY_ENVIRONMENT_VARIABLE
       value: “is awesome”
    image: macavaney/idabox
    imagePullPolicy: IfNotPresent
    resources:
      request:
        - cpu: 4
        - memory: 16Gi
  restartPolicy: Always
  serviceAccount: containerroot
```

A **restart policy** sets what to do if any of the containers die

After the container definitions, there is usually pod level configuration

The **permissions** the user running the container has is set via **Service Account**, your ‘containerroot’ service account… allows root permissions in the container (but not the host machine)

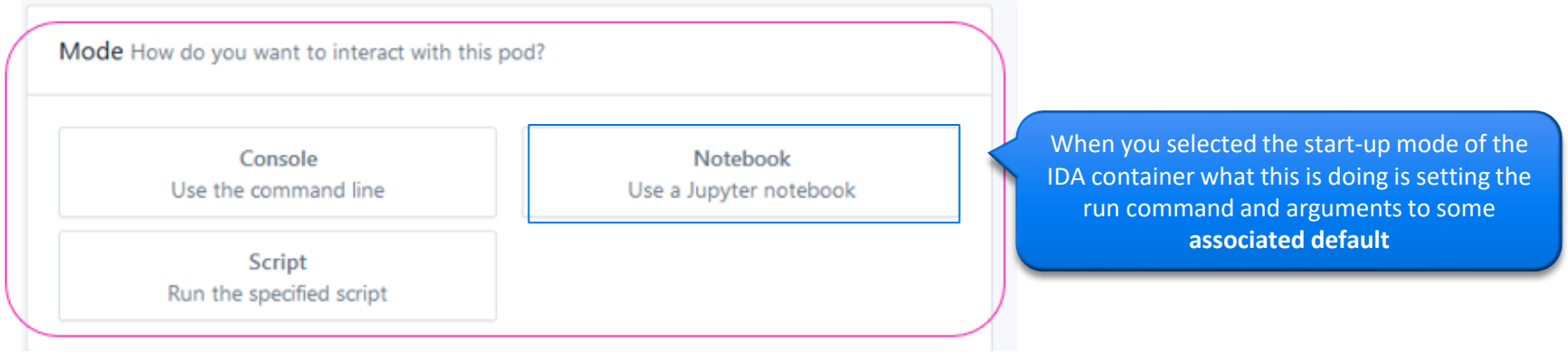


The Run Command

What happens on start?

Earlier I said that when a container starts it executes its run command

- This can be considered the primary process of a container, and is special in that if this process ever exits then the container is considered to be finished and will be destroyed



Note that the run command can be anything you could execute in a terminal, such as a shell script, python program or other executable

- Indeed, most containers set a shell script as the run command target

Overriding a Run Command

A default run command will be set by the container itself, but we can override it in the Pod…

Inside the container definition:

- command: sets the start command
- args: is a list of the arguments to pass to the specified command

This will start the container and have it **do nothing**, useful if you want to later open a terminal on a container and test different run commands

```
# This is an example Pod definition
apiVersion: v1
kind: Pod
metadata:
  name: mycontainer
  namespace: pgr24richard
  labels:
    app: idabox
spec:
  containers:
  - name: myapp
    image: macavaney/idabox
    imagePullPolicy: IfNotPresent
    …
    command: sleep
    args:
      - '365d'
    …
  serviceAccount: containerroot
```

Overriding a Run Command

A default run command will be set by the container itself, but we can override it in the Pod…

Inside the container definition:

- command: sets the start command
- args: is a list of the arguments to pass to the specified command

Here is an example for starting a **Jupyter** notebook server

```
# This is an example Pod definition
apiVersion: v1
kind: Pod
metadata:
  name: mycontainer
  namespace: pgr24richard
  labels:
    app: idabox
spec:
  containers:
  - name: myapp
    image: macavaney/idabox
    imagePullPolicy: IfNotPresent
    …
    command: jupyter
    args:
      - notebook
      - '--no-browser'
      - '--ip=0.0.0.0'
      - '--allow-root'
      - '--NotebookApp.token='
      - '--notebook-dir="/nfs/""
    …
```

Deployments

(Easier Management for Pods)

Problems with Pods

Launcher creates Pods directly on the cluster, however if you need to create Pods manually, you often want to iterate on configuration and be able to stop and start your pods

- Maybe I want to change the arguments passed to the start command, or alter the environment variables
- Maybe I am done with the pod for the moment and want to shut it down, but not delete the object

Both tasks are annoying if working with Pods directly, as they are immutable

- Once you create a Pod you cannot change its (YAML) configuration, you can only delete it

Hence, it would be nice if there was a way to create an editable 'template' for a Pod Object

Deployments

University of Glasgow

This is what Deployments are

- A Deployment is a template for creating one or more Pods
- A Deployment has a replication factor that specifies how many copies of the template you want running at any one time
  - You can set replication to 0 to shut down an application
- Deployments automatically 'roll-out' a new version of your pod(s) whenever you edit the deployment

```
# This is an example Deployment definition
apiVersion: v1
kind: Deployment
metadata:
  name: myNewApplication
  namespace: pgr24richard
  labels:
    app: idabox
spec:
  replicas: 1
  strategy:
    type: Recreate
    triggers:
      - type: ConfigChange
 containers:
 - name: container1
   image: macavaney/idabox
   resources:
     request:
       - cpu: 4
       - memory: 16Gi
  serviceAccount: containerroot
```

I want one Pod

The **strategy** specifies when and how to roll-out a new Pod

How: *Recreate* means delete the current one and then create a new one

When: *ConfigChange* means roll-out whenever this deployment is edited

Why Deployments?

University of Glasgow

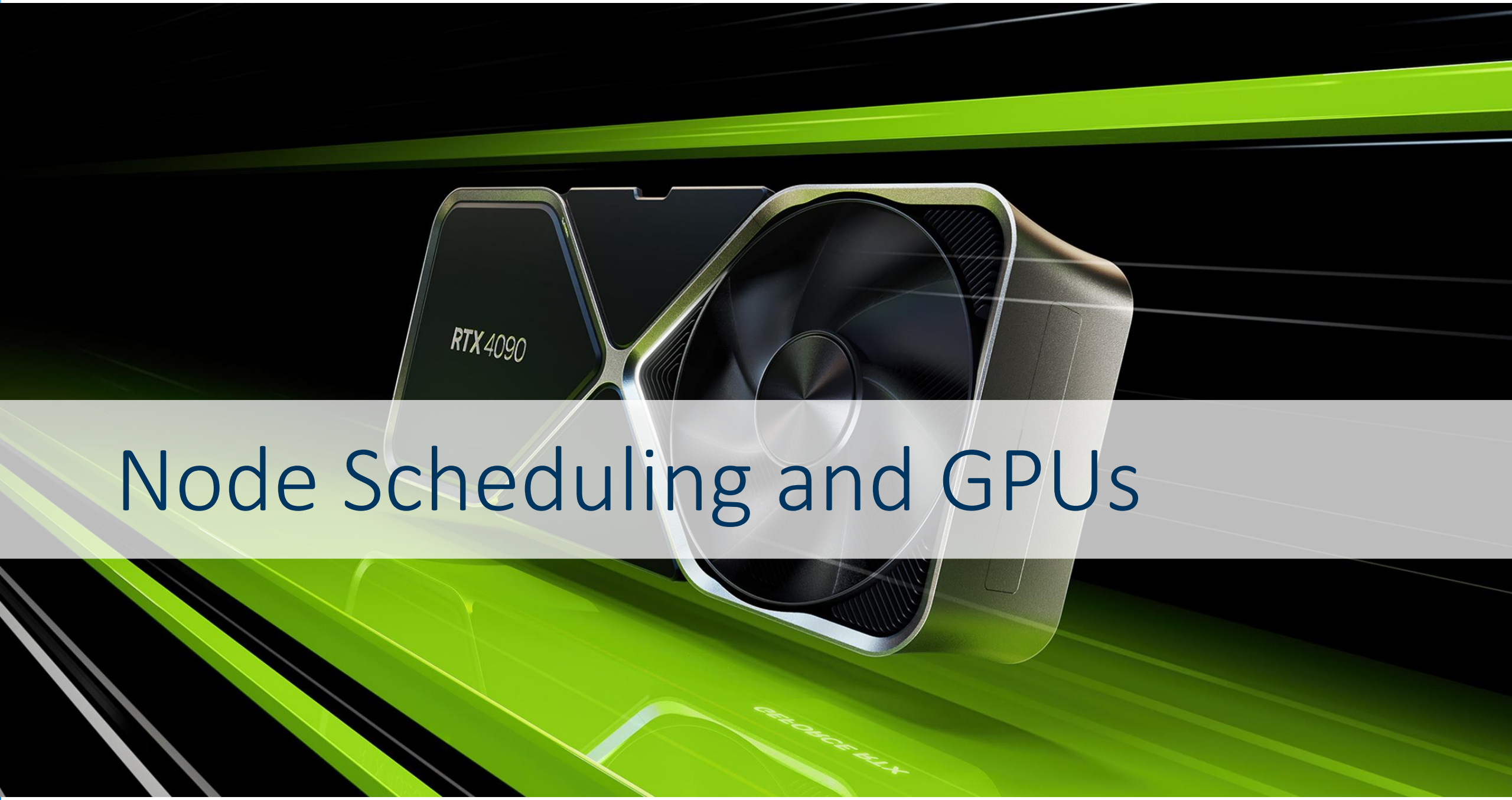
Deployments are a nice quality of life feature, since Deployments are editable

Richard's top uses:

- I want to shut-down a Pod, but also want to have the option of restarting it later:
  - Set replication factor to 0
- I have a new container I am testing, but its crashing:
  - Override the start command

```
command: sleep
args:
  - '365d'
```

- I have a rest API and need more capacity:
  - Set replication factor to >1
- I have a demo and want to have continuous integration
  - Use image version hooks for automatic roll-out of new versions (this is in the level 4 guide)



Node Scheduling and GPUs

GPU Requests

One of the main reasons that people use our cluster is such that they can get access to GPU cards for doing some form of AI workloads… so how can I request a GPU card?

Kubernetes does not provide a direct means to schedule your work on a GPU out of the box, and so we use a contextual scheduling mechanism based on how our cluster is configured

- Each compute node in our cluster is equipped with only one type of GPU card (i.e. they are homogenous)
  - Hence you might hear people talking about '4090 nodes' or 'A6000 nodes'
- Compute nodes have node labels, that specify what hardware the node has installed
- The Kubernetes scheduler allows us to select nodes for placement based on node labels

Node Selector

The nodeSelector field can be added to Pods and Deployments under the spec block of the object

- Only nodes that have the associated labels will be considered a valid scheduling target
- Labels are KEY: VALUE pairs

Currently, the valid nvidia.com/gpu.product values are

- NVIDIA-GeForce-RTX-4090
- NVIDIA-GeForce-RTX-3090
- NVIDIA-RTX-A6000

```
# This is an example Deployment definition
apiVersion: v1
kind: Deployment
metadata:
  name: myNewApplication
  namespace: pgr24richard
  labels:
    app: idabox
spec:
  nodeSelector:
    nvidia.com/gpu.product: NVIDIA-GeForce-RTX-4090
  replicas: 1
  containers:
  - name: container1
    image: macavaney/idabox
    resources:
      request:
        - cpu: 4
        - memory: 16Gi
        - nvidia.com/gpu: '1'
  serviceAccount: containerroot
```

Only schedule my pods on nodes with RTX 4090 GPUs

GPU Allocation

University of Glasgow

However, just because you get scheduled on a machine that has GPUs, does not mean you get allocated a GPU, to do that we need to tell the Nvidia Plugin we want one

We do this my specifying a GPU in the requests block

- nvidia.com/gpu: '1'

Number of GPUs requested

```
# This is an example Deployment definition
apiVersion: v1
kind: Deployment
metadata:
  name: myNewApplication
  namespace: pgr24richard
  labels:
    app: idabox
spec:
  nodeSelector:
    nvidia.com/gpu.product: NVIDIA-GeForce-RTX-4090
  replicas: 1
  containers:
  - name: container1
    image: macavaney/idabox
    resources:
      request:
        - cpu: 4
        - memory: 16Gi
      - nvidia.com/gpu: '1'
  serviceAccount: containerroot
```

Give my container one local GPU

Unshedulable

University of Glasgow

When Kubernetes determines that it needs to roll-out a new pod, the scheduler will check all of the machines that fit the specified criteria

In the example across only nodes that:

- Have the nvidia.com/gpu.product: NVIDIA-GeForce-RTX-4090 label
- Have at least 4 free CPU cores
- Have at least 16GB of unallocated RAM
- Have one unallocated GPU

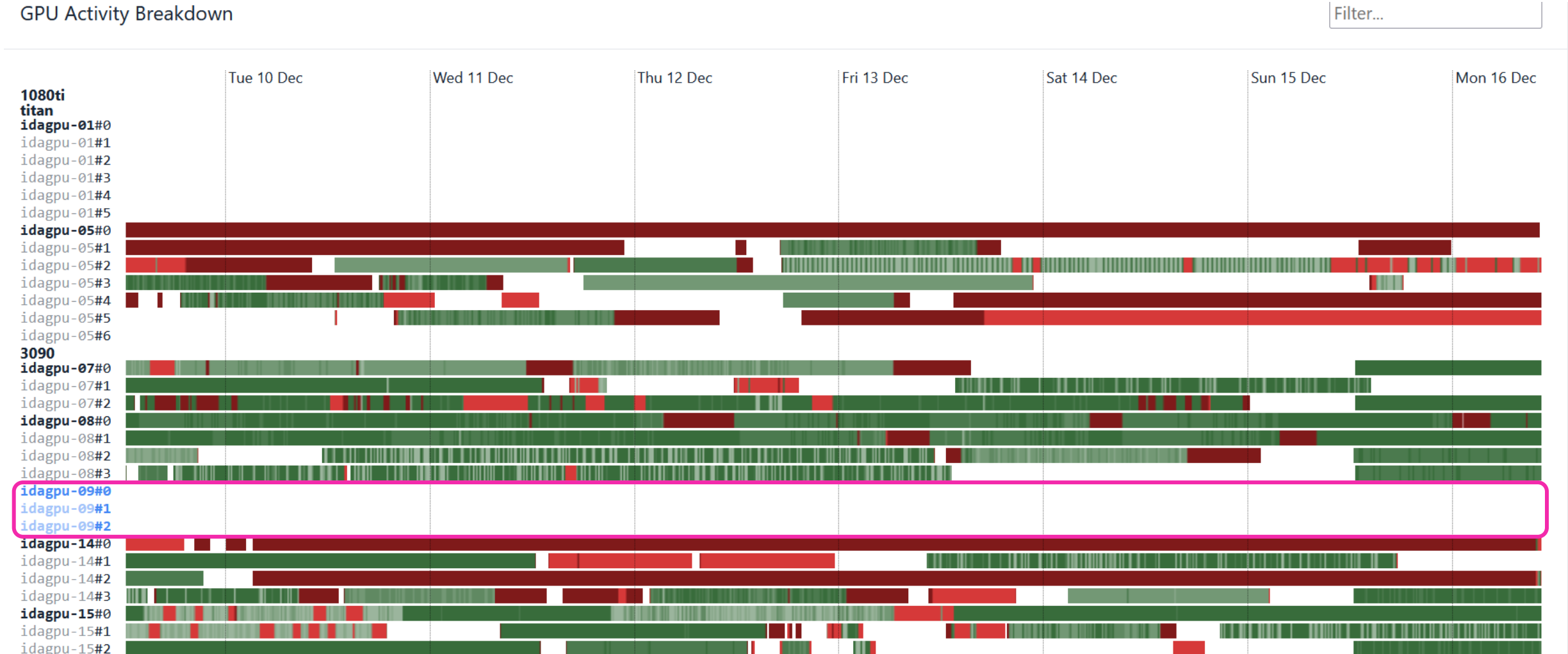
…will be considered. If no machines meet the criteria, the Pod will be marked as Unschedulable

- Kubernetes re-checks this constantly, i.e. the Pod will be scheduled when a node that meets the requirements becomes available

```
# This is an example Deployment definition
apiVersion: v1
kind: Deployment
metadata:
  name: myNewApplication
  namespace: pgr24richard
  labels:
    app: idabox
spec:
  nodeSelector:
    nvidia.com/gpu.product: NVIDIA-GeForce-RTX-4090
  replicas: 1
  containers:
  - name: container1
    image: macavaney/idabox
    resources:
      request:
        - cpu: 4
        - memory: 16Gi
        - nvidia.com/gpu: '1'
  serviceAccount: containerroot
```

How do I know if there are resources free?

In short, try it and see, you can also get a view on what GPUs are not in use via the usage dashboard (gpus in blue are in the new cluster)



Networking

Container Communication

University of Glasgow

For simple scripted work which only needs to execute a program that reads some data from disk, processes that data and then writes an outcome back to disk without any user input, the container does not need to communicate with other containers or the outside world

However, for most workloads, this is not true:

- Jupyter Notebook containers need external users to connect to their UI
- VSCode containers need to receive work from a client integrated development environment on the user's local machine
- Demos need to be accessible by external users
- Database containers need to be accessible by other containers that want to read/write data
- … and so on

For these scenarios, we need to tell Kubernetes **how we want communication to work**

Kubernetes Networking

University of Glasgow

The core challenge of networking in a cluster environment is how do we route information and requests between containers and the outside world

- Recall that when we register a Pod, we just state we want some containers to be running, but not where those containers should run
- This is intentional, as it means the user does not need to know the topology of the cluster, scheduling can handle this for you
- … but it means you don't know where your container is! (and hence cannot refer to its address)

To solve this problem, we can use Kubernetes Objects to define static addresses and then tie those addresses to our containers

- If we send a request to one of these static addresses, the request will be forwarded to the tied container(s)

There are two objects you need to know:

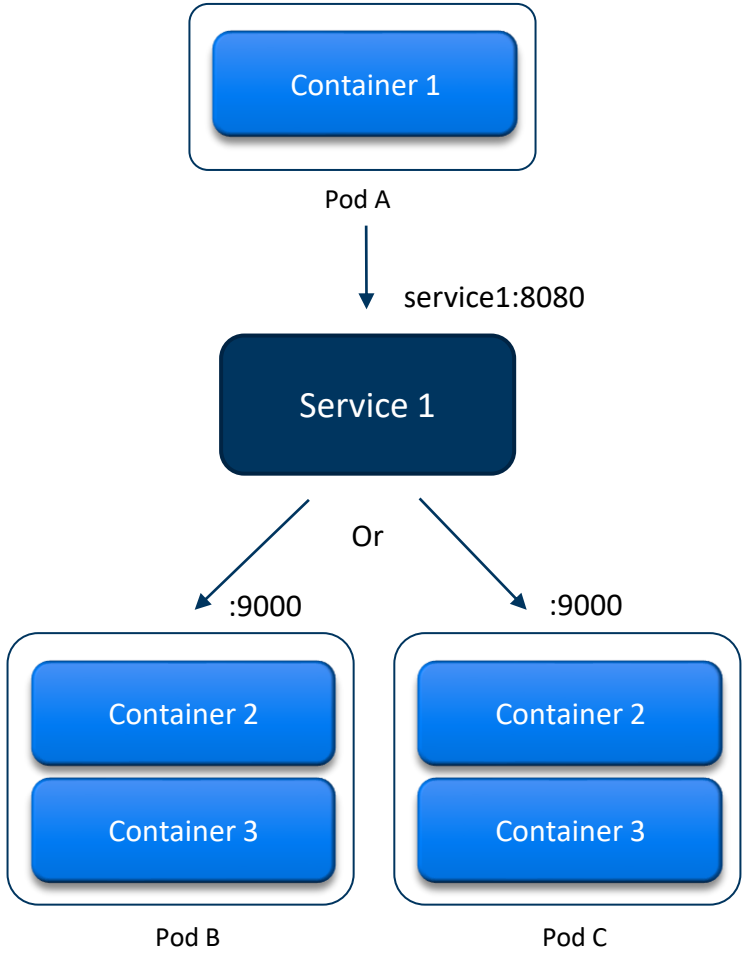
Services

Routes

Services

Service objects describe **cluster-internal addresses** that can be used to direct traffic to one or more containers

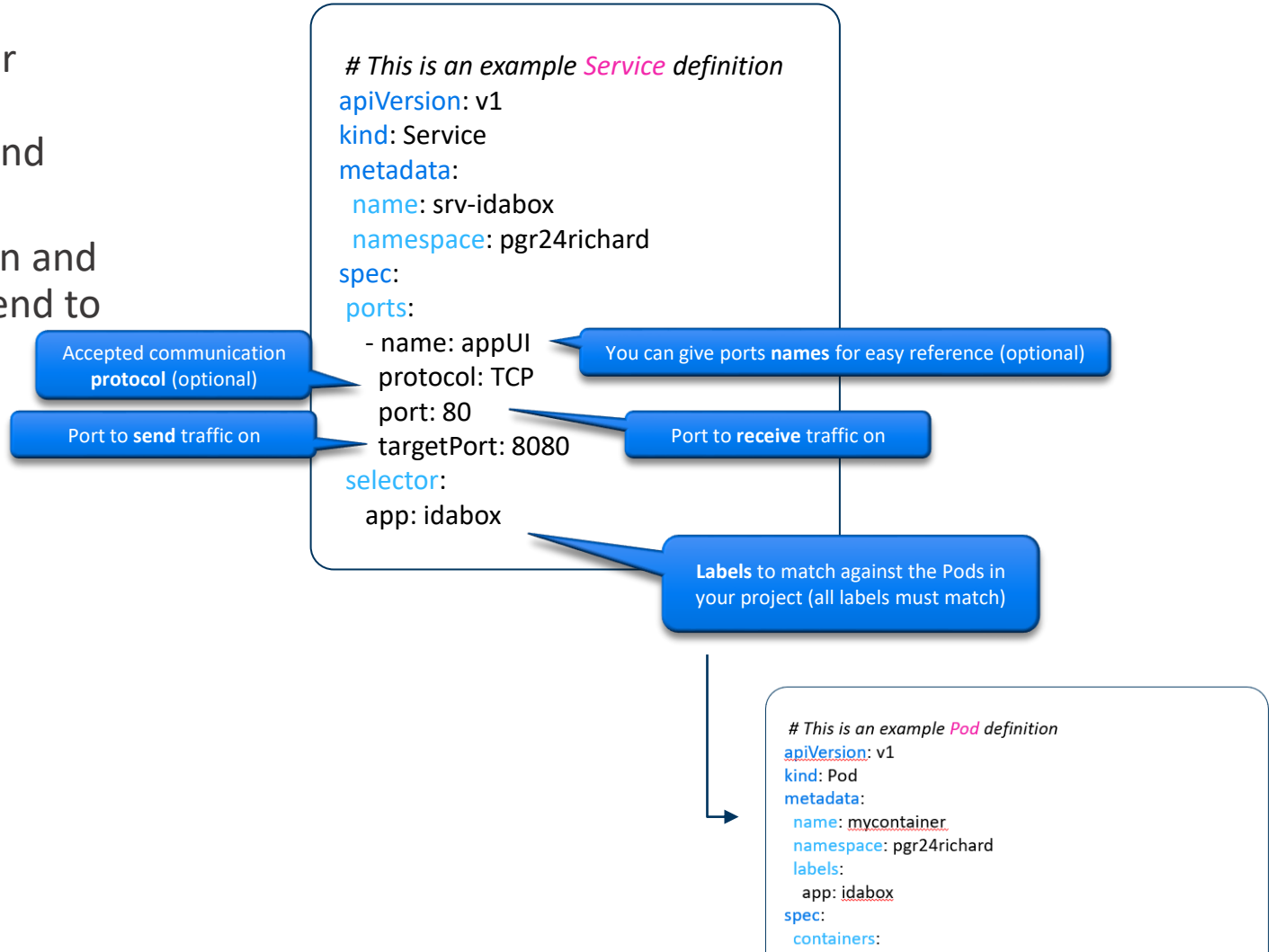
- Services are visible to all Pods across the network (within your project)
- Services have an internal hostname via which they can be reached
- Traffic directed at a service will be re-routed to designated Pod(s) based on rules and may change ports
- Can also act as a load-balancer for multiple Pods



Example: Service

A Service object stores two main pieces of information:

- Selector: This ties the service to one or more pods based on matching labels between the content of the selector and each Pod's label metadata
- Ports: A list of ports to accept traffic on and (if port switching) the target port to send to on the container




```
# This is an example Service definition
apiVersion: v1
kind: Service
metadata:
  name: srv-idabox
  namespace: pgr24richard
spec:
 ports:
   - name: appUI
     protocol: TCP
     port: 80
     targetPort: 8080
 selector:
   app: idabox
```

```
# This is an example Pod definition
apiVersion: v1
kind: Pod
metadata:
  name: mycontainer
  namespace: pgr24richard
  labels:
    app: idabox
spec:
  containers:
```

Example: Service

A Service object stores two main pieces of information:

- Selector: This ties the service to one or more pods based on matching labels between the content of the selector and each Pod's label metadata
- Ports: A list of ports to accept traffic on and (if port switching) the target port to send to on the container

In this example, if I sent a request to the URL 'srv-idabox:80' from within the cluster, that request would get sent to this service

- … and then forwarded to a pod with the 'app: idabox' label

```
# This is an example Service definition
apiVersion: v1
kind: Service
metadata:
  name: srv-idabox
  namespace: pgr24richard
spec:
 ports:
   - name: appUI
     protocol: TCP
     port: 80
     targetPort: 8080
 selector:
   app: idabox
```

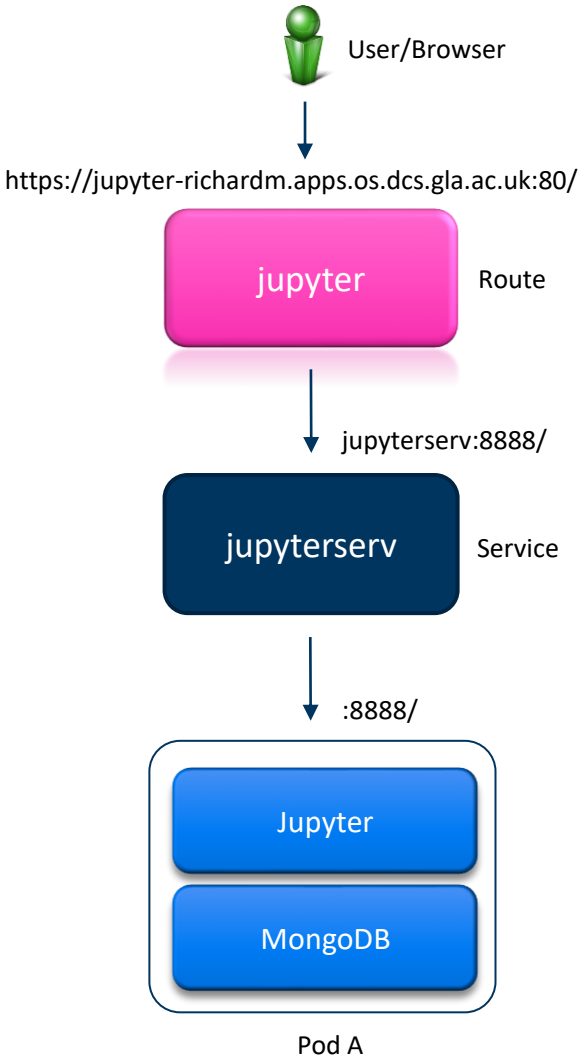
Routes

However, services have a limitation, in that they are only cluster-internal addresses – they cannot be accessed from the outside world

If we need to access a container from outside the cluster itself (the most common use-case being we have a user interface we want to access), then we need an **external URL**

This is the role of a Route

- A Route exposes a Service at a defined hostname for external access
- Routes are exposed via http://*.apps.os.dcs.gla.ac.uk:80/
- Where '*' is the hostname defined in the route, which is by default:
  - <routename>-<projectname>
  - E.g. http://jupyter-richardm.apps.os.dcs.gla.ac.uk:80/



Example: Route

A Route only needs one piece of information:

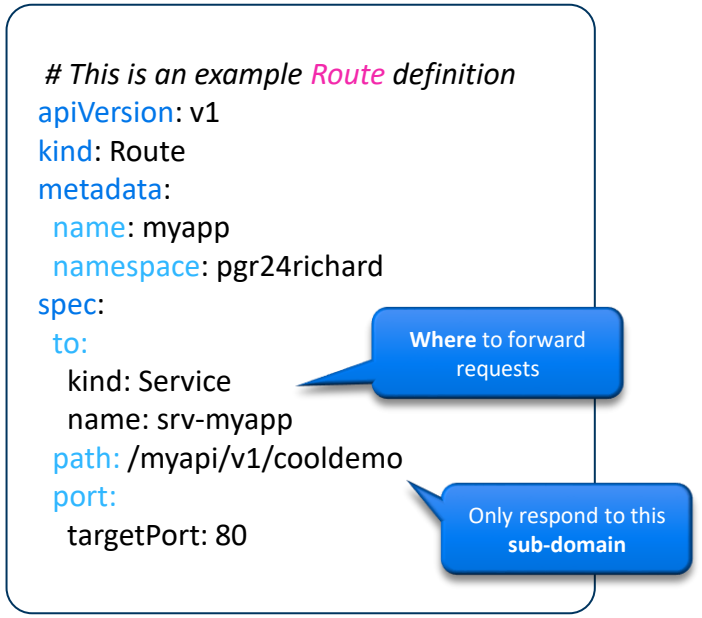
- to: This specifies the target of the route (where to send data), where you normally specify the target is a service and then give the name of the service to target

There are more advanced configuration options

- Path: Allows a route to only respond to particular sub-domains
- Port: Can allow for port switching at the Route
- Host: A custom URL can be specified (but still needs to end with apps.os.dcs.gla.ac.uk)

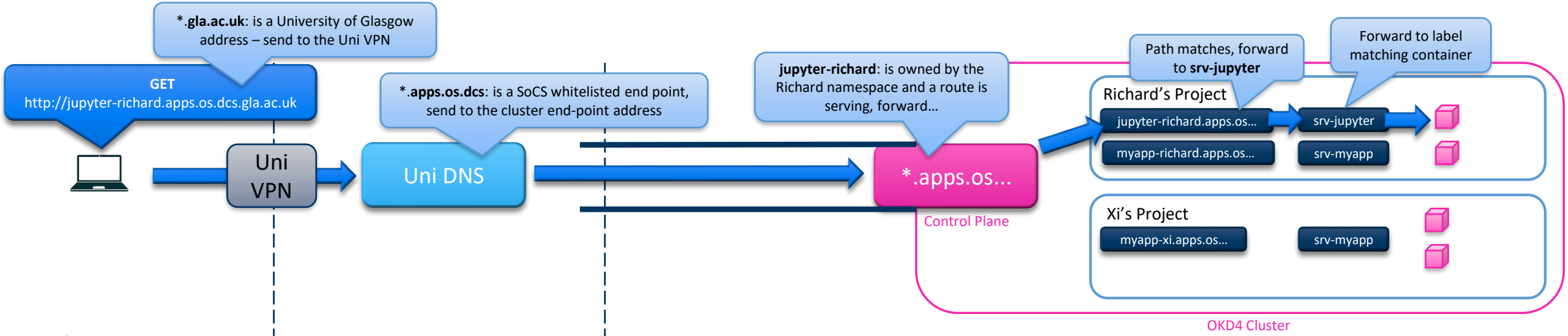
This example would create the route:

- https://myapp.pgr24richard.apps.os.dcs.gla.ac.uk/myapi/v1/cooldemo
- … which forwards requests to srv-myapp on port 80



Network Topology

I want to get a little technical to conclude this section, in case you find yourself needing to debug network problems … lets zoom out and look at the broader Uni network




Persistent Storage

Saving Your Data

As I have noted multiple times (its important!), any data stored within a container is deleted when that container is destroyed

- This is of course, bad…
- … and so we need a place to store our data such that it persists between containers

This is the role of persistent storage volumes

- Think of these as folders that can be attached to your containers
- Some of these folders you can only read, some you can read and write to
- These folders exist on an external storage medium separate from the machines that are doing the work

We create you a personal folder (volume) when we created your account

Persistent Volumes and Claims

University of Glasgow

Persistent storage is managed by two types of Kubernetes Objects

- Persistent Volume: Defines some available physical storage
- Persistent Volume Claim: Requests some persistent storage

When we created your account we created a pair of these for you

- The Persistent Volume (PV) defines a folder on our network file system (NFS) that is just for your use
  - You can't see this object, as it is stored in the cluster central object repository (outside your project)
  - You also can't (directly) create these, only a cluster administrator can
- The Persistent Volume Claim (PVC) allows a project to take ownership of the Persistent Volume
  - A PVC is a request for some storage, Kubernetes will search its available pool of available Persistent Volumes and allocate one that meets the specified requirements, a process known as Binding
  - For your personal folder, we just directly specify the target Persistent Volume by name

Example: PVC

University of Glasgow

As before, we define a persistent volume claim via a Kubernetes Object in YAML format

There are three main pieces of information

- accessModes: This acts as a matching criteria for the underlying persistent volumes and specifies the desired read/write properties
  - ReadWriteOnce: One pod can mount read/write
  - ReadOnlyMany: Multiple pods can mount as read only
  - ReadWriteMany: Multiple pods can mount as read/write
- resources: Another matching criteria, in this case a specification of properties of storage we want
- (optional) volumeName: An optional property that allows us to match by name to a persistent volume

```
# This is an example Persistent Volume
# Claim definition
kind: PersistentVolumeClaim
apiVersion: v1
metadata:
  name: richardmvol1claim
  namespace: richardmproject
spec:
  accessModes:
    - ReadWriteMany
  resources:
    requests:
      storage: 1Ti
  volumeName: richardsvolume
```

Mounting to a Container

University of Glasgow

To use our storage, we need to attach it to our container, in a process called mounting

There are two different pieces of information we need to specify to mount a persistent volume claim to a container

- volumes (spec-level): This is a mapping between reference names and a storage definition, in this case a Persistent Volume Claim
- volumeMounts (container-level): A mapping between a reference name and the location in the container file system you want the folder mounted

```
# This is an example Pod definition
apiVersion: v1
kind: Pod
metadata:
  name: mycontainer
  namespace: pgr24richard
  labels:
    app: idabox
spec:
  containers:
  - name: container1
    image: macavaney/idabox
    imagePullPolicy: IfNotPresent
    ...
    volumeMounts:
      - name: nfs-access
        mountPath: /nfs/
    ...
  serviceAccount: containerroot
  ...
  volumes:
      - name: nfs-access
        persistentVolumeClaim:
          claimName: richardsvolume
```

PVC Name

Dynamic Storage

It is worth noting that earlier I said that you ‘can not (directly) create persistent volumes’

- This is because if you create a new Persistent Volume Claim in the right way, Kubernetes will try and create a new persistent volume for you, in a mechanism referred to as dynamic storage allocation

You don’t need to know how to do this yet, as 90% of use cases are covered by the persistent volume claim we created for you, but we will cover this in a later tutorial

User Permissions

Who are you?

One of the areas where cluster users get confused is to do with access permissions

- This usually arises when a user finds that they don't have write permissions to a file they created earlier…

We are going to explore two important questions:

- Who is it running as?
- What permissions does that user have?

Multiple Distinct Users

When we create your user account, we create a cluster user, e.g. pgr24richard

- This user is a ‘project administrator’ for their project (which shares the user’s name)
- However, containers do not run as this user…

For legacy users, we may have also created a separate Unix storage account on the primary NFS file server, which again shares the same name

- This allows them to ‘ssh’ to the file server for uploading/downloading files
- However, containers do not run as this user either…

Instead, when a container starts, it gets assigned a virtual user account from the pool of user accounts on the host machine

- By default, this is a random user ID from a pre-set range
- If you set the ‘serviceAccount’ value to ‘containerroot’ it will assign you virtual user ‘0’ (root)

Why does this Matter?

There are a few key implications from this:

1. If not running as containerroot the user ID can change between instantiations of a container
   - This means that container A can create a file as user X, then a second container B could start with a different user Y, who would not have write permissions to that file (as they do not own it)
2. If you upload a file to the NFS through means other than a container, it will be owned by a user not in the virtual ID space the cluster uses, and so will not be editable
3. The permissions for the user (what they can do) is dependant on the permissions granted to the user ID the container is running as (not your user account)
   1. Hence a ‘default’ and ‘containerroot’ user can do different things

Service Account

When I say a ‘default’ or ‘containerroot’ user, what I am referring to is a Kubernetes Object called a Service Account

- This is a type of non-human account that, in Kubernetes, provides a distinct identity in a Kubernetes cluster
- The user the container runs as inherits the permissions of a specified service account

```
# This is an example Service Account definition
apiVersion: v1
kind: ServiceAccount
metadata:
  name: itsME
  namespace: pgr24richard
```

A **service account** has no special configuration, its power comes from roles and security contexts…

Our cluster manages permissions using a system called Role-based access control (RBAC)

- We can create a Service Account for a project…
- … and then assign it one or more roles that gives it API permissions
- … as well as security contexts that manage access control

The ‘containerroot’ service account is given a special power called **anyuid**, i.e. it can run as any (virtual) user, including (virtual) root!

What does this mean for files?

From a practical perspective, if reading and writing to your personal folder/NFS, you need to be mindful of your file permissions – the next container may be running as a different user

There are two main work-arounds for this behaviour

- If you always run as 'containerroot' then your user ID will be constant and hence minimise the chance of running into issues
- If working with multiple users, then set the group permissions for all files to be managed by the nfsnobody/nobody group from the command line
  - chmod –R 770 *
  - chgrp –R nfsnobody *

Make the file read/write/executable by group

Assign it the common group nfsnobody

What does this mean for permissions?

University of Glasgow

If you run as a user using the default service account, you will be executing as a generic non-root user

- You will not have write permissions to any system files
- Only the ~/ (home), /tmp/ and any mounted file systems will be editable
- Installations that need root will fail

If you run as containerroot, you will be running as a sudo root user in the container

- You an edit system files that are part of the container (but not host files)
- All files should be editable unless they are part of the host (e.g. some system drivers)
- Installations should work

User Overrides

University of Glasgow

There is one notable edge-case that is worth discussing that users have periodically run into, which is when a container creates its own user

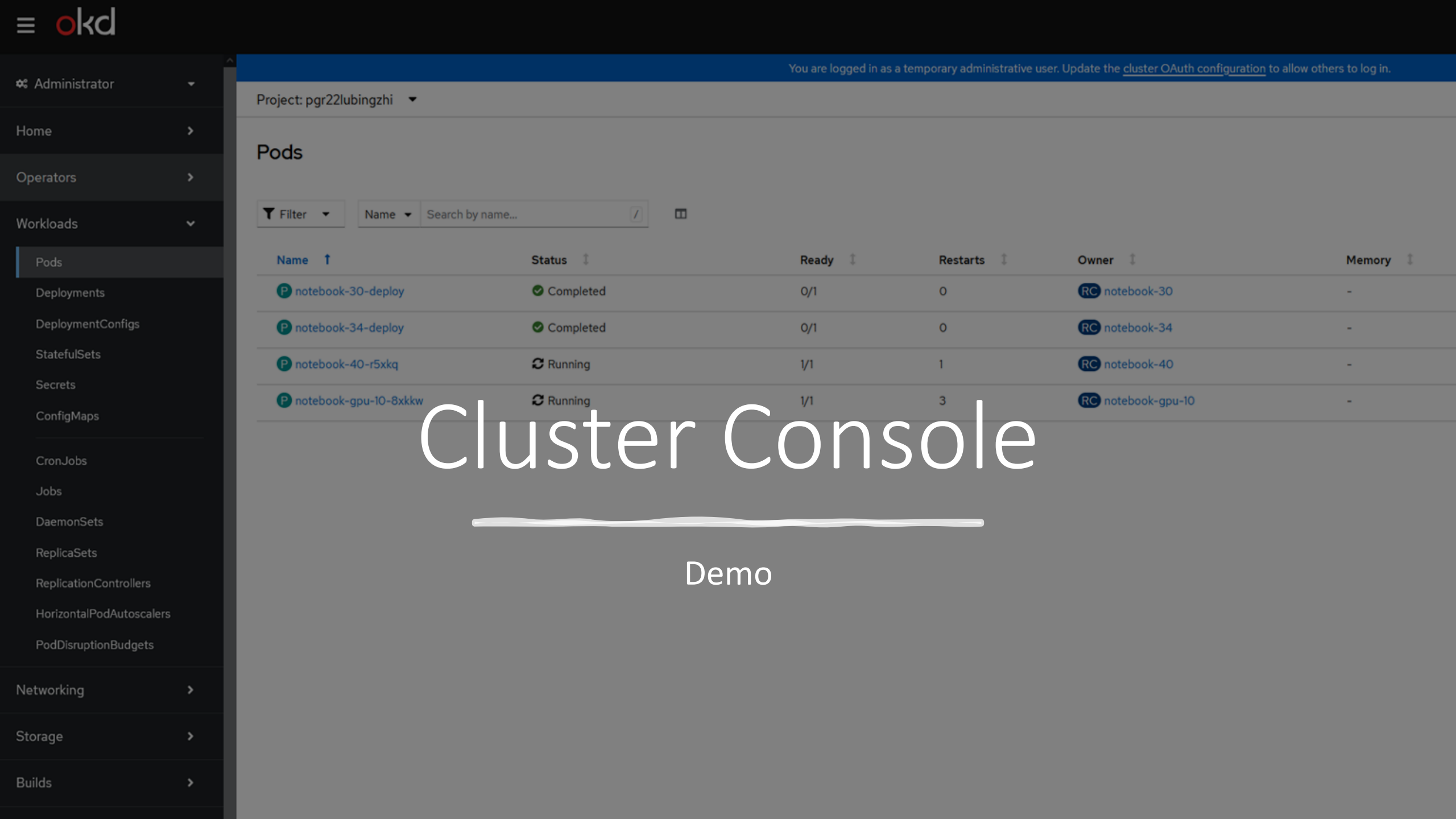
- This seems to be most common in applications like databases, which like to create a special user to run the database as

This causes issues because the new user created by the container does not inherit any of the permissions of service account, often leading to file access errors

- A workaround for this is to force the container to run as a specified virtual user in the Pod definition

```
# This is an example Pod definition
apiVersion: v1
kind: Pod
metadata:
  name: mycontainer
  namespace: pgr24richard
  labels:
    app: idabox
spec:
  containers:
  - name: container1
    image: macavaney/idabox
    imagePullPolicy: IfNotPresent
  serviceAccount: containerroot
  securityContext:
    runAsUser: 0
```

Run as root


okd
Administrator
Home
Operators
Workloads
Pods
Deployments
DeploymentConfigs
StatefulSets
Secrets
ConfigMaps
CronJobs
Jobs
DaemonSets
ReplicaSets
ReplicationControllers
HorizontalPodAutoscalers
PodDisruptionBudgets
Networking
Storage
Builds
You are logged in as a temporary administrative user. Update the cluster OAuth configuration to allow others to log in.
Project: pgr22lubingzhi
Pods
Filter
Name
Search by name...
Name
Status
Ready
Restarts
Owner
Memory
notebook-30-deploy
Completed
0/1
0
notebook-30
notebook-34-deploy
Completed
0/1
0
notebook-34
notebook-40-r5xkq
Running
1/1
1
notebook-40
notebook-gpu-10-8xkkw
Running
1/1
3
notebook-gpu-10
Cluster Console
Demo

Questions?

Dr. **Richard McCreadie**

Real-time IR, Machine Learning, Big Data Stream Processing, Evaluation

Richard.McCreadie@glasgow.ac.uk